Paper Review • arXiv:2611.14256

A Cross-Scenario Survey and Analysis of Self-Evolving Multimodal Agents

Self-Evolving Multimodal Agents: Why Mechanisms Fail to Transfer

MR MM Reasoning | MG MM Generation | GW GUI/Web Navigation | EM Embodied Manip.

Author: Jingjing Jiang | First submitted: 2026-11-10 | Homepage: jingjing12110.github.io/SE-MMAgents

One-Sentence Summary

Unifies four scenarios with a three-stage loop (experience generation → feedback acquisition → update consolidation), **locates each scenario by "rollout cap K_max × feedback-fidelity cap F_max"**, and treats each self-evolving mechanism as **a trade that borrows task-structure resources to boost a scarce factor**, structurally explaining why cross-scenario transfer is asymmetric and offering falsifiable portability verdicts and a measurement agenda.

① Framework

Three-stage loop EG / FA / UC

Each stage = product of two factors

② Position Matrix

K_max (how many attempts are affordable)

× F_max (how trustworthy feedback can be)

③ Mech.=Trade

Which factor it boosts · what resource it assumes

→ Portable / Conditional / Blocked / Redundant

④ Meas. Agenda

Verdicts not yet validated

Identifies controlled experiments that could refute them

Note: all portability verdicts are reasoned from single-scenario evidence; they are "testable preconditions", not empirical guarantees of transfer gains.

Problem: one scenario pair, two transfer directions, different obstacles

Majority voting: reasoning → robotics

In reasoning: consistency voting over sampled answers directly yields label-free rewards

On robots: action trajectories cannot be checked for equivalence, and physical rollouts incur execution and reset costs

⇒ No counterpart (lacks the resource of "equivalence-checkable outputs")

World model: robotics → reasoning

On robots: imagined rollouts bypass costly environment interaction

In reasoning: sampling directly from the policy is already cheap

⇒ Boosts a factor that is not scarce (redundant)

All four share one loop: explore → get feedback → consolidate lasting updates

Gap in prior surveys (Table I): they either do not analyze the self-evolution loop or treat it as scenario-agnostic; the closest work attributes verifiable feedback and retraining cost only at the domain level, not to specific mechanisms. This survey focuses on **cross-scenario portability**.

Contribution 1 Conditional definition & paradigm boundaries

Contribution 2 Stage-wise survey & positioning matrix

Contribution 3 Directed cross-scenario portability

Contribution 4 Falsifiable measurement agenda

Defn: autonomous loop (4 criteria) + self-reference = self-evolution

C1 Autonomy — After initialization, no human input in goals, feedback, updates or the action stream (pre-fixed task sets, checkers and judges count as bootstrap)

C2 Freshness — Each round's experience comes from the current policy, not a static dataset

C3 Persistence — Updates are written to parameters, long-term memory, tools or interfaces, persisting across episode boundaries

C4 Recursion — The loop runs more than one round; each round's output becomes the next round's agent

Differentia: Self-Reference (SR)

The agent itself implements at least one meta-operator: goal sampler σ, feedback function V or evolution operator f. An autonomous learning loop without self-reference is a "fixed loop", used as the control group throughout.

Provenance matrix Γ: rows = meta-operators; cols = implementer (h human / b bootstrap / e env. / a agent)

SimpleVLA-RL (fixed loop)

	h	b	e	a
σ	0	1	0	0
V	0	1	0	0
f	0	1	0	0

VisPlay (self-evolving)

	h	b	e	a
σ	0	1	0	1
V	0	0	0	1
f	0	1	0	0

Criteria read directly off the matrix: C1 ⇔ column h all 0; SR ⇔ column a not all 0. VisPlay's internal Questioner poses questions and scores its own answers by voting, so rows σ and V contain a.

Scope: only 4 of 16 adjacent paradigms self-evolve by definition

5

Outside the genus

- Self-training (on classifiers)
- Test-time adaptation
- Test-time scaling (non-persistent)
- Self-correction (within one turn)
- Interactive imitation learning (expert actions enter the data)

2

Never satisfy SR

- Continual learning (task stream pre-fixed)
- RLHF (reward model fits human preferences)

5

Split by SR

- Self-Improvement
- RLVR
- Agentic RL
- Prompt / workflow optimization
- Experiential learning

4

Self-evolving by def.

- Self-Play （σ）
- Automatic curriculum learning (σ)
- Self-rewarding / label-free RL (V)
- Recursive self-improvement (f)

Contested cases hinge on a: dropping tied groups in a batch is not SR; allocating later rollouts by task success rate is;

Evolution operator: who writes? TextGrad, guided by a separate fixed model (GPT-4o) → fixed loop; ExpeL insights, STaR rationales written by the acting model → self-evolving.

Unlike prior surveys: fixed-dataset RL with env./rule rewards (e.g., SimpleVLA-RL) is not self-evolving without agent meta-operators.

Framework: loop = 3 stages; stage efficacy = product of 2 factors

① Experience Generation EG

Cap K_max

Meta-operator: goal sampler σ

$$Q = K_k \times E[I]$$

- K_k: rollouts per goal, bounded by cap K_max = B / E[c(τ)]
- I: return variance (p(1−p) for binary rewards); I≈0 for mastered or unreachable goals

Governs: speed & breadth

② Feedback Acquisition FA

Cap F_max

Meta-operator: feedback function V

$$Q = (2F_v - 1) \times L_v$$

- F_v: agreement between feedback ranking and true-return ranking, bounded by cap F_max
- L_v: fraction of environment actions that receive credit

Governs: alignment with true R

③ Update Consolidation UC

Meta-operator: evolution operator f

$$Q = S_\zeta \times G_\zeta$$

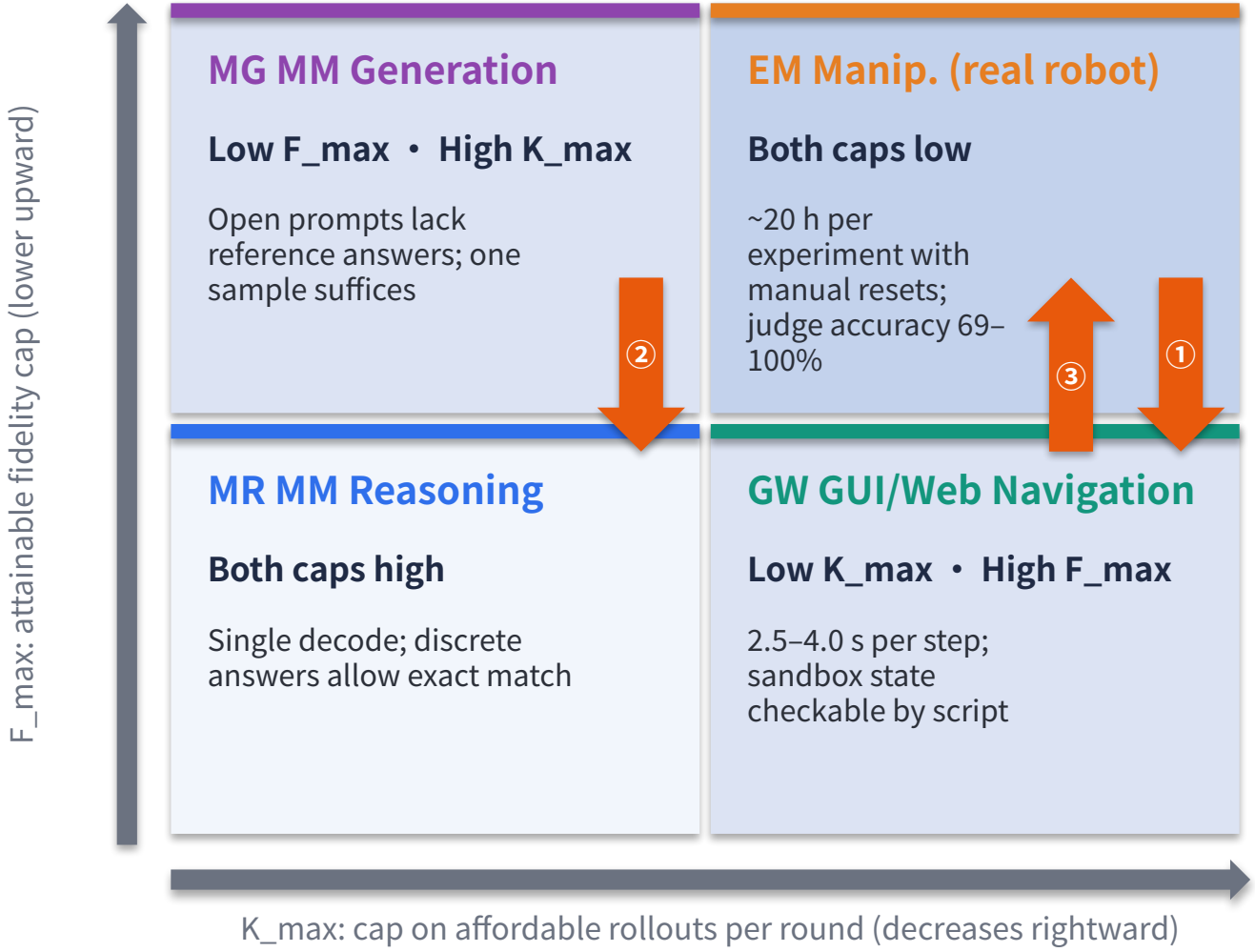
- S_ζ: fraction of gains retained after subsequent rounds
- G_ζ: fraction of gains generalizing to unsampled goals

Determines: retention & generalization

↺ **Cross-cycle coupling:** agent edge (update → next-round rollout policy) + goal edge (this round's trajectories & scores → next-round goal sampling)

Why only K_max and F_max as axes? Only they are both exogenously set by the scenario's task structure and not substitutable by the other factor in the same stage. The information-gain cap depends on backbone coverage; a shortfall in L_v can be offset by higher F_v; S_ζ and G_ζ have no scenario-fixed cap.

Position matrix: one quadrant per scenario, different binding caps



Three shifts (orange arrows)

① Simulation: success predicates raise F_max, group sampling relaxes K_max → EM moves toward GW (most embodied loops actually learn in simulation)

② Verifiable subfamilies (OCR-scored prompts, programmatically rendered charts) → MG moves toward MR

③ Live websites / free-form goals lack state scripts and rely on screenshot judges → GW moves toward EM

Key point

The two off-diagonal cells—cheap but exploitable generation, costly but verifiable navigation—are combinations a single “difficulty ranking” would merge.

Limitations: positions are inferred from each scenario’s reported numbers; backbone pretraining coverage is a confounder.

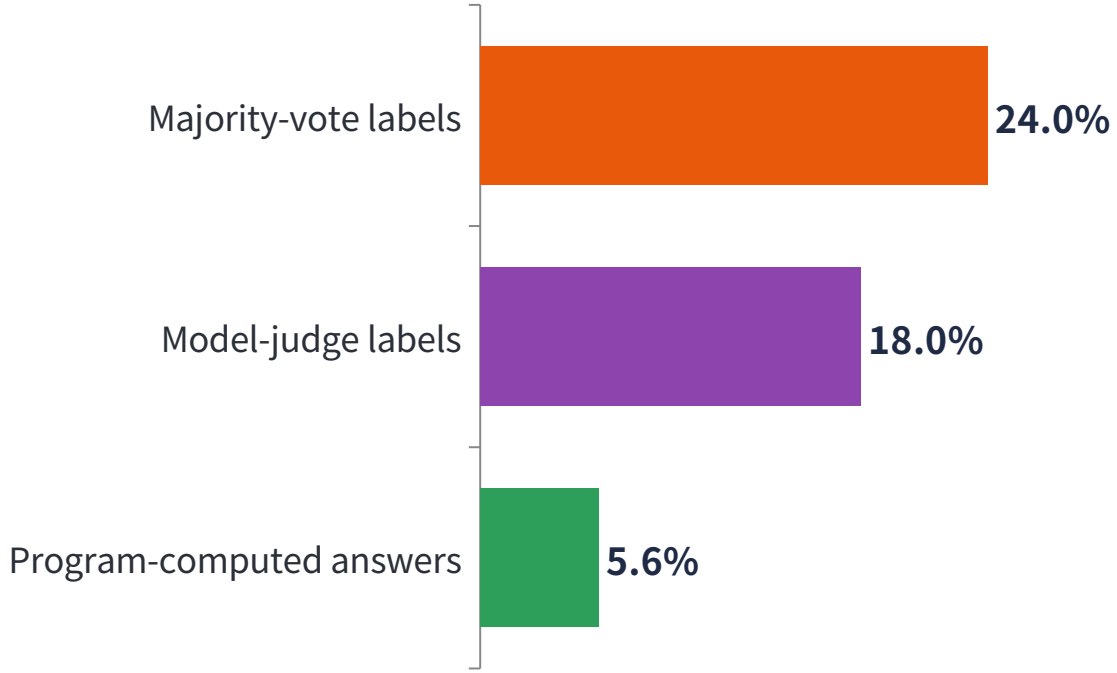
Each scenario’ s bottleneck lies in a different stage and factor

Stage / Metric	MR MM Reasoning	MG MM Generation	GW GUI/Web Navigation	EM Embodied Manip.
EG Generation	Quality (saturated)	Quality	Quantity	Both (real robot)
FA Feedback	Achieved fidelity F_v^(k)	Fidelity cap F_max	Credit localization L_v	Both (real robot)
UC Consolidation	Retention	Retention	Generalization	Generalization
Main failure mode	Long-horizon drift	Compounding reward hacking	Distribution shift	Forgetting & dist. shift
Reported group size K_k	4–10	3–16	8–64	8
Cost per rollout	One decode	One sample or tool chain	Per-step env. latency	Per-step latency + episode reset

How to read: identical numbers do not imply identical bottlenecks. MG and GW report overlapping group sizes, but MG sampling is cheap and reversible, limited by the information in a fixed prompt pool; GW episodes are latency-bound, limited by K_max itself. Likewise, MR and GW both have high F_max, but MR has L_v = 1 and is limited by fidelity decline from self-reference, whereas GW is limited by credit localization over long episodes.

MR: self-reference bottleneck—keyless self-label fidelity drops

Retained-label error, same 500 vetted Qs (VQS)



Program-computed answers are correct on 94.4% of retained questions; consensus and self-judgment fall below this level already in round 1.

Consistency voting assumes answers are equivalence-checkable and the pretrained prior concentrates on the correct answer—the latter set by the backbone, not the task.

72.0→61.0

VisPlay pseudo-label accuracy
(3 rounds)

64.5→37.5

Video-Zero pseudo-label accuracy
(5 rounds)

4–6

EvoLMM self-consist. optimum; 12: consensus collapse

30.61→44.16

VisPlay, Questioner trained 1 round; untrained Challenger only 33.77

Interpretation

Neither cap is tight; fixed loops (e.g., M-STAR) use answer keys for exact matching, while self-evolving loops give up keys to pose their own questions, paying the cost in feedback. Note: cross-round accuracy drops are confounded with rising question difficulty.

Multimodal generation: the fidelity ceiling binds even before self-reference

0.63→0.95

Flow-GRPO (fixed loop) gain on GenEval—the checked prompt pool saturates; yet human raters judge its image quality below the base model

r = −0.17

Correlation of ManimAgent's frozen reviewer with blind human ratings of 37 instructional videos

8.6 vs 0.5

RSI in UMMs: gain (points) from training on execution-verified charts vs. frozen-initial-model self-selection (same requests & steps)

72%→60%

RSI in UMMs: exact-text retention on the old vocabulary after a vocabulary swap—the only retention measured across rounds

Self-verifiers differ in policy dependence, and so in bias pattern (four groups in Table IX)

Policy-updated verifiers

Generation confidence, understanding branch → bias moves with the policy, prone to compounding reward hacking (SOLACE, IRIS degrade within one training run)

Coevolving/lagged verif.

Previous-round LVLM, co-trained critic, moving-average teacher → slower drift

Frozen self-verifiers

Frozen initial model or frozen backbone both judges and writes → fixed bias, but judge and writer share the same bias

External/program verif.

Frozen external scorers, program execution → cut the agent edge, but cover only checkable subfamilies

GUI/Web nav.: costly episodes; memory value depends on the reader

2.5–4.0 s

Per-step latency (UI-TARS-2 VM); an episode has up to 25 steps, lasting tens of minutes

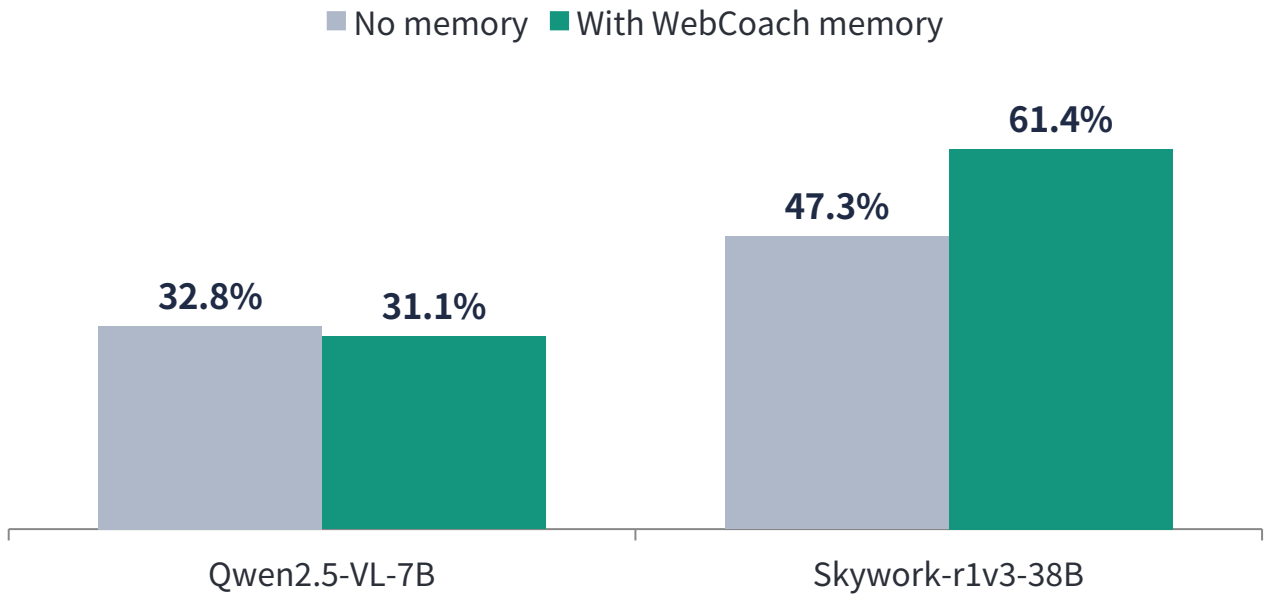
8–64

Group size bought in parallel; DigiRL ⩽64 emulators

11.3% / 18.1%

Initial success rates of SEAgent and UI-Genie—$p(1-p) \approx 0$ on hard goals

Memory helps or hurts by reader (WebVoyager success %)

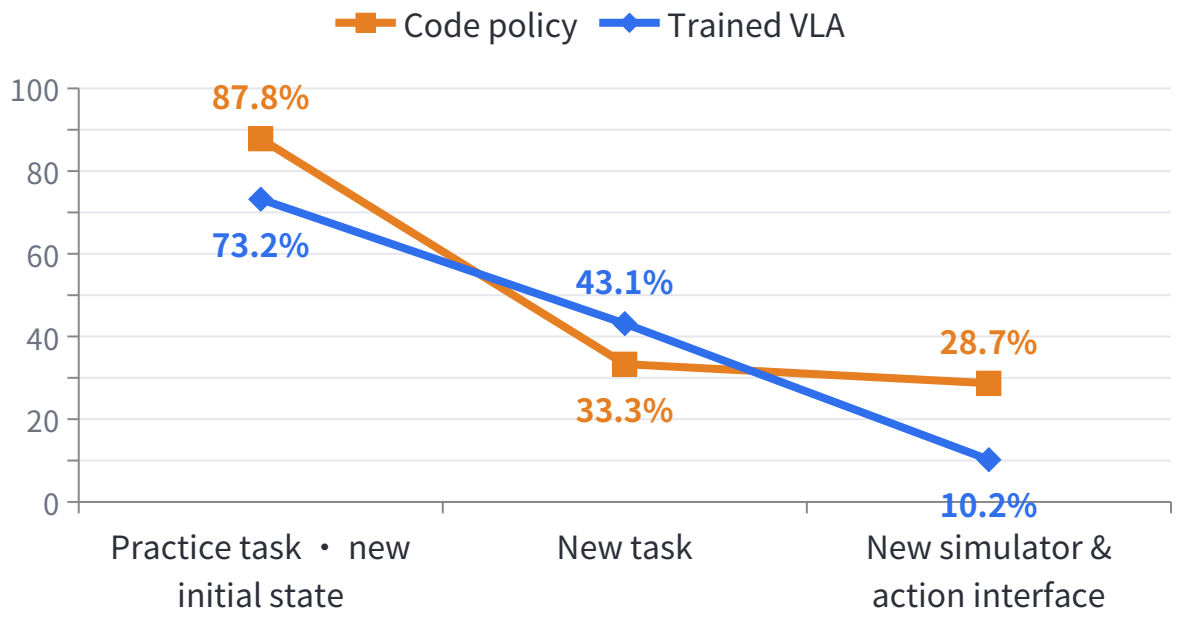


Key point

Three ways to raise information: reuse (memory/skills), success-rate reweighting (DART-GUI, MobileRL), new-goal generation (SEAgent, ACuRL). Refitting reward models on own trajectories for step-level credit makes fidelity move with the policy. EvoSkill-GUI's same-backbone critic misjudged 28 of 143 audited failures.

Embodied: success falls off-practice; checks beyond writer are key

SEAR: success falls as tests move further from practice



Real-robot cost: Self-Improving EFMs needs ~20 hours with manual resets per real experiment; in Find Something, 6.58% of 456 autonomous episodes end in states only humans can recover; external VLM judges take 2.6–16.9 s per query, with real-robot judge accuracy of 69%–100%.

Scale and Selection: checks beyond writer?

40%→0%

Accept every revision (20 rounds)

51%→67%

champion-challenger selection (held-out success rate)

Another counterexample: one SEAR run stored a self-calibrated but faulty placement verifier in memory and ended at only 4.6%, vs. 15.5% and 23.6% for two sibling runs that did not save it.

Key point

On real robots, both caps are equally tight for fixed and self-evolving loops; self-reference changes "who poses, scores and writes", not the cost or attainable fidelity of physical rollouts. The farther the check is from the writer, the more gains are retained.

Five cross-cycle degradations: surface, spread and fix sites may differ

Degradation mode	Cross-cycle mechanism	Surfaces in	Propagation path	Main mitigation site
① **Compounding reward hacking**	Per-round tolerable fidelity defects in FA are repeatedly amplified into directional bias	UC	agent edge (enters FA when V reads the agent)	Tighten F_v in FA, or shrink update step size
② **Distribution shift**	Rollouts drift outside V’ s calibration support; errors become extrapolation	FA	agent edge, environment path	Recalibrate V in FA, or resample in EG
③ **Forgetting & plasticity loss**	Goal distribution shifts; parameter writes interfere	UC	Environment path, goal edge	Replay early goals in EG, or isolate/reset parameters
④ **Curriculum collapse**	σ shrinks goal support to mastered regions; E[I] decays	EG	goal edge, or agent edge into σ	Inject exploration into σ, or score goals by difficulty
⑤ **Model collapse**	Self-generated data narrows the policy and erases tails	EG	agent edge	Accumulate real or external data rather than replacing it

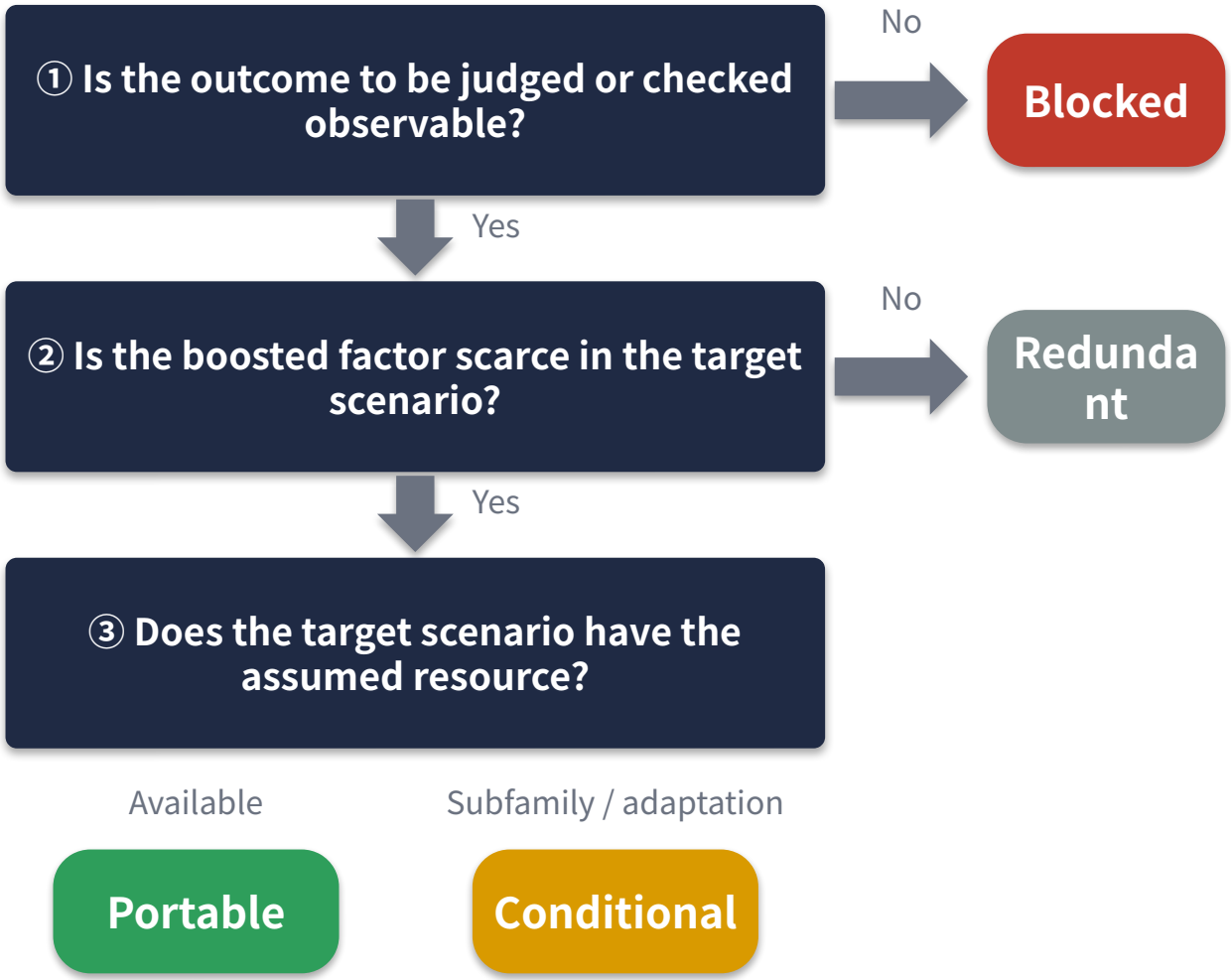
Most typical mismatch: reward hacking and forgetting both surface in UC, but better fixes lie upstream—tightening fidelity in FA, replaying in EG. Long-horizon drift is the first three compounding within one task family, not a sixth mode. Current loops run only a few rounds, so these mechanisms are mostly inferred from within-run and fixed-loop evidence.

Mechanism as trade: boost a scarce factor by assuming a resource

Mechanism	Entry	Boosts	Assumed resource	Cost
Success-conditioned allocation	σ	E[I]	Fixed task set with checkable outcomes (T)	Coverage of deliberately down-weighted tasks
Agent-posed goals	σ	E[I]	Checkable outcomes or equivalence-checkable outputs (T)	Forgoes answer keys/scripts; label accuracy 72%→61%
Program-checkable goal families	σ	E[I]	Programs, simulators or scripted scene checks (T)	Out-of-family coverage
World-model imagined rollouts	V	K beyond K_max	Learnable short-horizon dynamics (T)	Model error lowers F_v; video degrades after ~300 frames
Self-consistency voting	V	Key-free F_v	Equivalence-checkable outputs (T) + prior concentrated on correct answer (M)	K rollouts per goal; label errors
Model self-judgment	V	Checker-free F_v	Generation–verification gap (M)	Fidelity moves with policy
Generation-confidence reward	V	Checker-free F_v	Generator confidence (M)	Fidelity tied to the weights being updated
Refitting verifiers on own trajectories	V	L_v / F_v on free-form goals	Labeled seeds or reference set	Fidelity moves with policy
Frozen self-derived estimator	V	L_v / F_v on free-form goals	Calibration on demos or initial model	Fixed bias outside training support
Memory / skill reuse	f	G_ζ, E[I] on hard goals	Writer’ s generation–verification gap (M) or online checks (T)	Exposure to writer’ s errors

(T) task-structure resource (M) agent-model capability unmarked = artifact authored before the loop. **Only (T) resources are unevenly distributed across scenarios and thus determine verdicts; (M) and artifacts are assumed available in all target scenarios.**

Verdict rules: observability → scarcity → resources → four outcomes



Uneven task-structure resources across scenarios

Resource	MR	MG	GW	EM
Equivalence-checkable outputs	✓	✗	✗	✗
Checkable outcomes	✓	Program-checkable subfamily only	✓ (sandbox)	Sim only
Resettable, parallel rollouts	✓ Free	✓ Free	Paid instances	Sim only

Two caveats

- Observability alone determines blocking; it even blocks mechanisms that rely only on model capability.
- Having a resource ≠ using it: only "online checks" running inside the loop lower the trade cost. Task streams, open prompts and self-selected practice lack online checks—a self-built verifier saved by one SEAR run erred on 91% of later rollouts.

Cross-scenario asymmetry: verdict type depends mainly on the target

MR → EM (real robot) — Conditional

Voting and goal mechanisms become "conditional"; "blocked" on contact-determined subfamilies (on FailBench no judge exceeds 0.60 balanced accuracy there; best overall is 0.77)

EM → MR — Redundant

World models and estimators are "redundant" on reasoning problems with answer keys or program-computed answers (cheap rollouts, $L_v = 1$)

MG → GW — Redundant

Generation-side judges are "redundant" on navigation goals with online checks, but self-judgment remains portable to unchecked task streams

GW → MG — Conditional

Navigation's self-built goals are "conditional" on open prompts: they need image-based questions, rendered charts or external judges

Entering MR meets only "redundant"; entering MG and real robots mainly yields "conditional" goal mechanisms and voting; where outcomes are unobservable (GW backend state, real-robot contact assembly), always "blocked".

Scarcity decides Redundant

Task-structure resources decide Portable / Conditional

Observability alone decides Blocked

Eval.: flawed benchmarks/judges mislead self-evolving agents more

43.6%

Sphinx-X-MoE's MMMU score without images exceeds its language backbone by 17.9 points

>90%→0.0%

VLA scores on LIBERO → under LIBERO-PRO perturbations

+14.9

MemGUI-Bench: M3A's gain from retries alone

62.9–72%

Accuracy of the strongest multimodal judge on fixed outputs

48.1%

Share of reward hacking on VQA policies

17.7

Max deviation of GenEval from human judgment (points)

Why riskier for self-evolution: loops trained with the evaluation scorer (e.g., Reflection-VLA on LIBERO) write memorized goals and scorer errors into the agent. Self-evolution benchmarks are scarce; only Flex-RSI covers both reasoning and generation.

Three claims currently untestable

Gains come from self-evolution

Budget-matched non-evolving baselines (incl. manual robot resets) + loop-unseen held-out refs

Scenario positions are correct

Needs rollout costs in unified units and F_max measured on optimized outputs

Portability verdicts hold

Budget-matched unadapted-transfer controls in the target; conditional cases add adapted arms

Four open questions: a yes/no answer directly rewrites the verdicts

Reference construction

Can substitute references be built for unobservable outcomes without humans in the loop?

Yes → blocked becomes conditional; No → blocking is intrinsic to the outcome. The four existing routes (output-to-answer conversion, programs, world models, VLM judges) all succeed only partially.

Self-derived verification

Once the policy leaves the support where the verifier was formed, can a verifier of any dependence level stay consistent with R?

Yes → reliability becomes a choice of dependence level; No → low F_{max} in generation and real robots becomes a structural limit.

Cross-carrier transfer

Can skills, harnesses and policies take effect across readers, platforms and embodiments (incl. sim→real) without new experience?

So far, cross-platform zero-shot transfer appears only in the fixed loop GUI-HARVEST (GPT-5 +13.87); none is reported across the sim-to-real boundary.

Safety under autonomy

Without human veto, can the loop preserve safety properties across cycles?

Evidence shows safety declining along every self-modification path: SEAgent's unsafe completion rate on RiOSWorld rose 39.5%→55.3%.

Summary & Assessment

Three takeaways

1. To judge whether a self-evolving mechanism transfers to a new scenario, ask not how "hard" it is, but which factor the target lacks, which task-structure resources it has, and whether outcomes are observable.
2. In reasoning, where both caps are loose, the bottleneck is self-reference itself—forgoing answer keys in exchange for self-generated labels that drift with the policy.
3. Every self-write (skills, memory, harness) needs checks beyond the writer; the farther from the writer, the more gains are retained.

Strengths

- The provenance matrix makes "what counts as self-evolution" decidable item by item
- Uses fixed loops as controls to separate scenario limits from self-evolution costs
- Offers falsifiable predictions and a concrete measurement agenda, honestly flagging evidence limits
- Clear lessons for embodied research: contact tasks may be unobservable; VLM judges have structural flaws

Limitations

- Verdicts are derived from rules, unsupported by transfer experiments
- Positions come from single-scenario estimates, entangled with backbone priors
- Provenance labels and taxonomy axes set by a single author, after bottlenecks were identified
- No coverage of audio, touch or force feedback; dense terminology hurts readability